

Extended Abstract

Motivation Large Language Models struggle with complex reasoning tasks requiring real-time information retrieval and computation. Current approaches using Retrieval-Augmented Generation and multi-agent tool usage suffer from fragmented decision-making and suboptimal credit assignment, while prompt-based methods lack systematic optimization through training. Existing Reinforcement Learning with Verifiable Rewards (RLVR) methods haven’t effectively integrated external tools like search engines and code execution into unified RL-driven policies, limiting the development of adaptive, knowledge-intensive AI systems.

Method This work introduces CodeSearch-RL, the first training-based single-policy reinforcement learning framework that integrates both search engines and sandboxed code execution environments within a unified optimization objective. The core innovation lies in modeling search engines and code execution tools directly within the RL environment, enabling end-to-end training of a single policy capable of dynamically interleaving reasoning, retrieval, and execution actions. The method adapts Group Relative Policy Optimization (GRPO) with specialized loss masking for external tool outputs, ensuring that optimization focuses exclusively on LLM-generated tokens while avoiding destabilization from externally returned content.

Implementation The system operates through structured iterative rollouts with maximum action budgets to prevent infinite loops. The framework employs explicit token annotations to manage different phases of task-solving, clearly distinguishing reasoning steps (`<think> </think>`), search engine queries (`<search> </search>`), retrieved information (`<information> </information>`), code generation (`<code></code>`), and execution outcomes (`<output></output>`). Loss masking is applied to all external tool outputs during policy gradient and KL-divergence calculations for training stability. The reward system uses simple rule-based exact match evaluation, avoiding neural reward model complexity while allowing the model to explore any valid reasoning path leading to correct answers.

Results Our experiments demonstrate that while integrating tools like search and code execution provides an initial performance boost, effective utilization of these tools requires reinforcement learning. Preliminary results show that simple architectures (e.g., Executor-only) outperform more complex ones, as untrained components like planners can degrade performance. In our main experiments, the Qwen-2.5-3B-base model without GRPO training failed to leverage tools, yielding near-zero accuracy. However, with GRPO training, the model achieved substantial gains—26% EM on in-distribution data, 24% on SimpleQA, and 4% on GAIA-text—highlighting improved reasoning and generalization. We also observed that base models adapt better to GRPO than instruct-tuned ones, which plateaued or degraded in performance.

Discussion Our study shows that language models, even as small as 3B parameters, can be effectively trained via reinforcement learning to integrate search and code execution, yielding significant gains in complex reasoning tasks. We find that training a base model with GRPO leads to more stable and effective learning than using instruction-tuned models, which appear less adaptable—supporting the view in many recent works that supervised fine-tuning can limit exploration during RL. Our tool integration via a code sandbox offers a flexible, extensible architecture capable of supporting diverse external APIs or domain-specific tools. Looking ahead, we see strong potential for applying this agent framework to scientific research, where the ability to search, compute, and analyze mirrors the workflow of discovery—making it a promising direction for accelerating progress in science.

Conclusion CODESEARCH-RL demonstrates that single-policy reinforcement learning can effectively unify retrieval, computation, and reasoning without multi-agent coordination overhead. This approach has significant implications for scientific discovery and research automation, showing that training-based tool integration substantially enhances LLM performance beyond baselines

CODESEARCH-R1: Developing a Multi-Role Tool-Integrated Reinforcement Learning Agent

Fan Nie

Department of Electrical Engineering
Stanford University
niefan@stanford.edu

Harper Hua

Department of Computer Science
Stanford University
shuohua@stanford.edu

Mingjian Jiang

Department of Computer Science
Stanford University
jiangm@stanford.edu

Abstract

We present the first training-based single-policy reinforcement learning framework that integrates search engines and code execution environments for complex reasoning tasks. Unlike existing approaches that rely on rigid multi-agent architectures or static prompting systems, our method trains a unified policy to dynamically interleave reasoning, information retrieval, and computational execution within a single RL optimization loop. Our framework employs explicit token annotations to manage different phases of task-solving, uses loss masking to maintain training stability over external tool outputs, and adopts a streamlined outcome-based reward strategy focusing on final answer correctness. By modeling external tools directly within the RL environment and enabling end-to-end training of tool usage strategies through GRPO, our approach represents a superior performance than fragmented multi-agent systems that suffer from suboptimal credit assignment. Our experiments show that our method can boost the performance of base model from 1% to 26% on knowledge question-answering and mathematical benchmarks, validating the effectiveness of our integrated training approach for tool-augmented language model reasoning.

1 Introduction

Large Language Models (LLMs) have shown remarkable proficiency in natural language understanding and generation (Hendrycks et al., 2020; Clark et al., 2018). However, they face significant challenges when handling complex reasoning tasks, particularly those requiring real-time external information retrieval (Wei et al., 2022; Jin et al., 2024a). Traditional retrieval-augmented generation (RAG) (Gao et al., 2023; Lewis et al., 2020) and tool-usage approaches (Schick et al., 2023; Yao et al., 2023) partially address these limitations. Nevertheless, these approaches rely predominantly on multi-agent architectures or static prompting, lacking end-to-end training integration and adaptability necessary for dynamic, knowledge-intensive scenarios.

Recent advancements propose Reinforcement Learning with Verifiable Rewards (RLVR) methods to enhance the reasoning and decision-making capabilities of LLMs by allowing them to learn from interactive feedback and outcome-based rewards (Guo et al., 2025; Kumar et al., 2024). Despite these promising directions, effectively integrating external tools such as search engines and code execution environments into a single RL-driven LLM policy remains unexplored. Current approaches exhibit limitations: they either (1) employ rigid multi-agent architectures (Lin et al., 2025; Nagpal

et al., 2025) that suffer from fragmented decision-making processes and suboptimal global credit assignment, or (2) rely exclusively on prompt-based scaffolding systems that lack the systematic optimization achievable through comprehensive training methodologies (Ma et al., 2025).

To address these critical gaps, we introduce CODESEARCH-R1, the first training-based single-policy RL framework that integrates both search engines and a sandboxed code execution environment. Specifically, our framework presents the following contributions:

- Unlike prior approaches, our framework models search engines and code execution tools directly within the RL environment, enabling end-to-end training of a unified policy capable of dynamically interleaving reasoning, retrieval, and execution actions. We employ retrieval token masking to maintain training stability, ensuring robust optimization.
- Our system uses explicit token annotations to manage different phases of complex task-solving, clearly distinguishing reasoning steps (`<think>` `</think>`), search engine queries (`<search>` `</search>`), retrieved information (`<information>` `</information>`), code generation, and execution outcomes. This structured iterative approach supports dynamic decision-making and robust task management.
- We adopt a streamlined, outcome-based reward strategy emphasizing final answer correctness, minimizing complex intermediate rewards. Through rigorous experimentation, we demonstrate that this minimalistic reward effectively guides the LLM to leverage retrieval and execution tools meaningfully.

2 Related Works

2.1 Retrieval-Augmented Language Models

Retrieval-Augmented Generation (RAG) enhances LLMs by fetching external documents to ground generation, improving factuality and up-to-date knowledge (Lewis et al., 2020; Gao et al., 2023). Iterative schemes such as IRCOT (Trivedi et al., 2022) improve multi-hop reasoning but often require extensive annotations and can introduce irrelevant context (Jin et al., 2024a). To mitigate spurious retrievals, recent work applies entailment-based reranking or trains models to ignore distracting passages (Jiang et al., 2023). Ultra-long-context LMs (e.g. with 128K tokens) offer an alternative by processing whole documents directly, but at significant computational cost (Li et al., 2024). Unlike these modular or static pipelines, our framework treats retrieval as a learnable action within a unified RL policy, enabling the model to decide dynamically when and what to retrieve to optimize scientific reasoning.

2.2 Tool Integration and Execution in LLMs

Augmenting LLMs with external tools—calculators, code interpreters, APIs—extends their problem-solving beyond raw text. Toolformer trains LLMs to invoke APIs via supervised annotation (Schick et al., 2023), while ReAct interleaves reasoning and tool calls in prompts (Yao et al., 2023). Modular multi-agent systems (e.g. AutoGen) separate planning and execution roles but rely on fixed decision rules (Huang and Chang, 2022), and domain-specific agents like CALMS can even control lab instruments through code (Prince et al., 2024). However, most approaches use handcrafted or supervised signals to trigger tools, lacking end-to-end learning. In contrast, our method embeds tool-use decisions into a single RL loop, allowing the agent to learn—from trial and error—both when to call each tool and how to integrate its outputs for complex scientific tasks.

2.3 Reinforcement Learning for LLMs

Reinforcement learning has been key to aligning LLMs (RLHF with PPO) (Ouyang et al., 2022; Schulman et al., 2017) and to direct preference methods like DPO (Rafailov et al., 2023). More recently, RL has been harnessed to boost reasoning: DeepSeek-R1 fine-tunes LMs on math benchmarks with outcome-based rewards (Guo et al., 2025), Satori applies chain-of-thought RL for proof generation (Shen et al., 2025), and AReaL scales reasoning RL via asynchronous rollouts (Fu et al., 2025). These studies optimize either text-generation or discrete action policies separately. Our CODESEARCH-R1 framework is the first to unify retrieval, tool execution, and textual reasoning within a single RL policy, delivering end-to-end training for integrative, tool-driven scientific reasoning.

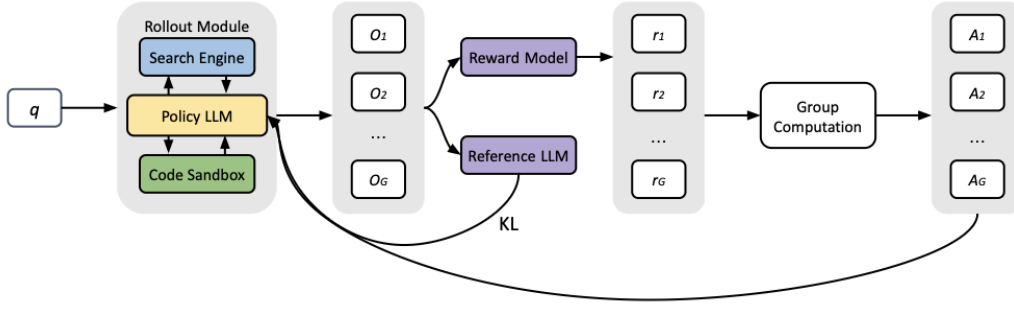


Figure 1: Method Overview.

3 Method

In this section, we introduce the detailed methodology for CODESEARCH-R1, our single-agent framework that integrates planning, reasoning, and execution with external tools. Our approach extends upon search engine retrieval by incorporating a code execution module, enabling the model to solve complex problems requiring both information retrieval and computational execution in a unified training-based framework. Unlike previous methods, which rely either on static pipelines or multi-agent frameworks lacking end-to-end training, our approach allows seamless interleaved interactions with external tools (search engines and code interpreters) within a single RL policy. We detail our approach, covering: (1) the formulation of Reinforcement Learning (RL) to utilize both a search engine and a code interpreter; (2) the generation process with interleaved, multi-turn tool use; (3) the training template that guides the model; and (4) the design of our outcome-based reward model.

3.1 Reinforcement Learning with a Search Engine and Code Interpreter

We formulate the problem of learning to use external tools within a unified RL objective. The goal is to train a single policy, π_θ , that learns to strategically call upon a search engine, $\mathcal{S}$, and a code interpreter, $\mathcal{C}$, to maximize its performance on complex tasks. The objective function is defined as:

$$\max_{\pi_\theta} \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_\theta(\cdot | x; \mathcal{S}, \mathcal{C})} [r_\phi(x, y)] - \beta \mathbb{D}_{\text{KL}} [\pi_\theta(y | x; \mathcal{S}, \mathcal{C}) || \pi_{\text{ref}}(y | x; \mathcal{S}, \mathcal{C})], \quad (1)$$

where π_θ is the policy LLM we are training, and π_{ref} is a reference LLM. The input samples, x , are drawn from a dataset $\mathcal{D}$, and y represents the full generated output, which includes the model’s reasoning, calls to external tools, and the final answer.

Unlike previous works that are either not training-based or rely on multi-agent systems, our framework explicitly trains a single policy to perform interleaved reasoning and tool execution, which can be denoted as $\pi_\theta(\cdot | x) \otimes \mathcal{S} \otimes \mathcal{C}$. This integrated approach allows the model to fluidly transition between generating reasoning steps, querying for external knowledge, and executing code to perform calculations, all within a single, coherent policy. This stands in contrast to prior RL methods that primarily generate text from the policy LLM $\pi_\theta(\cdot | x)$ alone.

Our method builds on the powerful policy gradient RL algorithm Group Relative Policy Optimization (GRPO), adapting it to handle the unique challenges of multi-tool use.

3.1.1 Loss Masking for External Tool Outputs

In CODESEARCH-R1, a complete generation, or "rollout," contains tokens generated by the LLM (reasoning, queries, code snippets) as well as tokens returned by external tools (search results, code execution outputs). Optimizing the LLM’s generated tokens is essential for improving its ability to reason and correctly utilize the tools. However, applying loss gradients to the externally returned tokens—which the model has no control over—can destabilize training.

To address this, we adopted **loss masking for all external tool outputs** from Search-R1 (Jin et al., 2025). The policy gradient and KL-divergence are computed *only* over the tokens generated by the

policy LLM, π_θ . This ensures that the optimization process focuses exclusively on what the model can learn: how to reason and when and how to call the search engine and code interpreter.

3.1.2 GRPO with Search and Code Execution

To enhance training stability and eliminate the need for a separate value function, we also adapt Group Relative Policy Optimization (GRPO). For each input x , GRPO samples a group of G responses $\{y_1, \dots, y_G\}$ from the policy. The objective function is:

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}_{x \sim \mathcal{D}, \{y_i\}_{i=1}^G \sim \pi_{\text{old}}(\cdot|x; \mathcal{S}, \mathcal{C})} \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{\sum_{t=1}^{|y_i|} I(y_{i,t})} \sum_{t=1: I(y_{i,t})=1}^{|y_i|} \min \left(\frac{\pi_\theta(y_{i,t}|x, y_{i,<t}; \mathcal{S}, \mathcal{C})}{\pi_{\text{old}}(y_{i,t}|x, y_{i,<t}; \mathcal{S}, \mathcal{C})} \hat{A}_{i,t}, \right. \right. \\ \left. \left. \text{clip} \left(\frac{\pi_\theta(y_{i,t}|x, y_{i,<t}; \mathcal{S}, \mathcal{C})}{\pi_{\text{old}}(y_{i,t}|x, y_{i,<t}; \mathcal{S}, \mathcal{C})}, 1 - \epsilon, 1 + \epsilon \right) \hat{A}_{i,t} \right) - \beta \mathbb{D}_{KL} [\pi_\theta || \pi_{\text{ref}}] \right], \quad (2)$$

Here, the advantage $\hat{A}_{i,t}$ is computed from the relative rewards of outputs within the group, avoiding the need for a learned value model. The KL-divergence term is added directly to the loss, and the loss masking $I(y_{i,t})$ is applied to both the policy gradient and the KL-divergence calculation, ensuring optimization only occurs on LLM-generated tokens.

3.2 Generation with Multi-turn Search and Code Execution

The rollout process enables the LLM to dynamically interact with its environment, which consists of the user query, a search engine, and a code interpreter. The model operates in an iterative loop of reasoning and tool execution.

The LLM is prompted to first reason about the problem. When it determines that it needs external information, it generates a search query encapsulated between special tokens, `<search>` and `</search>`. The system intercepts this, queries the search engine $\mathcal{S}$, and appends the results to the context, enclosed in `<information>` and `</information>` tokens. Similarly, if the model needs to perform a calculation or execute a program, it generates a code snippet enclosed in `<code>` and `</code>` tokens. This snippet is passed to the sandboxed code interpreter $\mathcal{C}$, and the execution output (e.g., a numerical result, string, or error message) is returned to the model within `<output>` and `</output>` tokens.

This cycle of reasoning, searching, and coding continues until the model determines it has sufficient information and provides a final answer, enclosed in `<answer>` and `</answer>` tokens. The complete workflow is detailed in Algorithm 1.

3.3 Training Template

To facilitate the learning of this complex behavior, we provide the model with a simple, structured prompt. As shown in Table 1, this template instructs the model on the syntax for tool use without imposing any constraints on *how* or *when* to use them. The template encourages an iterative "reason-act" cycle.

This minimalist approach ensures that the model learns its own strategies for planning and tool selection through the RL process, rather than being overly guided by handcrafted rules. This allows us to observe the emergent problem-solving behaviors that arise purely from optimizing the final outcome reward.

3.4 Reward Modeling

A key component of our framework is the reward function, which provides the learning signal for the policy. We deliberately adopt a simple, rule-based reward system focused exclusively on the **final outcome**. This approach avoids the complexity and potential biases of training a separate neural reward model. For factual question-answering or mathematical reasoning tasks, the reward is determined by the correctness of the final extracted answer:

$$r_\phi(x, y) = \text{EM}(a_{\text{pred}}, a_{\text{gold}}), \quad (3)$$

Algorithm 1 LLM Response Rollout with Multi-Turn Search & Code Execution

Require: Input query x , policy model π_θ , search engine $\mathcal{S}$, code interpreter $\mathcal{C}$, maximum action budget B .

Ensure: Final response y .

```
1: Initialize rollout sequence  $y \leftarrow \emptyset$ 
2: Initialize action count  $b \leftarrow 0$ 
3: while  $b < B$  do
4:   Initialize current action LLM rollout sequence  $y_b \leftarrow \emptyset$ 
5:   while True do
6:     Generate response token  $y_t \sim \pi_\theta(\cdot \mid x, y + y_b)$ 
7:     Append  $y_t$  to rollout sequence  $y_b \leftarrow y_b + y_t$ 
8:     if  $y_t$  in [</search>, </code>, </answer>, <eos>] then
9:       break
10:    end if
11:  end while
12:   $y \leftarrow y + y_b$ 
13:  if <search> </search> detected in  $y_b$  then
14:    Extract search query  $q \leftarrow \text{Parse}(y_b, \text{<search>}, \text{</search>})$ 
15:    Retrieve search results  $d = \mathcal{S}(q)$ 
16:    Insert  $d$  into rollout  $y \leftarrow y + \text{<information>}d\text{</information>}$ 
17:  else if <code></code> detected in  $y_b$  then
18:    Extract code snippet  $c \leftarrow \text{Parse}(y_b, \text{<code>}, \text{</code>})$ 
19:    Execute code and get result  $r = \mathcal{C}(c)$ 
20:    Insert  $r$  into rollout  $y \leftarrow y + \text{<output>}r\text{</output>}$ 
21:  else if <answer> </answer> detected in  $y_b$  then
22:    return final generated response  $y$ 
23:  else
24:    Ask for rethink  $y \leftarrow y + \text{"My action is not correct. Let me rethink."}$ 
25:  end if
26:  Increment action count  $b \leftarrow b + 1$ 
27: end while
28: return final generated response  $y$ 
```

where a_{pred} is the answer extracted from the model’s final `<answer>` `</answer>` block and a_{gold} is the ground-truth answer. We use exact match (EM) for simplicity, though other rule-based metrics could be substituted.

By focusing solely on outcome-based rewards, we empower the model to explore and learn any valid path of reasoning and tool use that leads to a correct answer. We do not use process-based or format-based rewards, as we found the model learns the required structural format effectively through the RL process itself. This design choice simplifies the training setup and reduces the computational overhead associated with training and maintaining an additional reward model.

4 Experimental Setup

4.1 Preliminary Experiments

To inform the design of our main experiments, we first conducted a preliminary study to compare the efficacy of different agent architectures. This initial experiment utilized two datasets: DeepScaleR (Luo et al., 2025), a collection of approximately 40,000 unique math problem–answer pairs, sourced from competitions like AIME (1984–2023), AMC (pre-2023), Omni-MATH, and Still, designed to train and evaluate reinforcement-learned language models on advanced mathematical reasoning, and MedQA (Jin et al., 2020), a large-scale, multiple-choice open-domain medical question-answering benchmark sourced from professional medical board exams (e.g., USMLE, Chinese equivalents), covering over 60,000 questions across English, simplified Chinese, and traditional Chinese with accompanying reference materials for comprehension. For these tests, we used gpt-4.1-mini for DeepScaleR and gpt-4o-mini for MedQA. We compared four distinct agent architectures: a

Table 1: Training Template for CODESEARCH-R1

You are answering a question that could require multiple steps to solve, combining search, coding and reasoning.

IMPORTANT: You MUST follow this EXACT format for EVERY response:

1. Start with `<think>` your reasoning process `</think>`
2. Use `<search>` `</search>` and `<code>``</code>` for searches and calculations.
3. When code returns results, they will appear between `<information>` and `</information>` tags
4. Use as many `<think>` `</think>` / `<search>` `</search>` / `<code>``</code>` blocks as needed to solve the question
5. ALWAYS end with `<answer>` your final answer `</answer>`

RULES:

- NEVER skip the `<think>` `</think>` step or provide a final answer without `<answer>` `</answer>` tags
- ALL code must be inside `<code>``</code>` tags
- Code sessions are isolated - you cannot use variables from one code block in another
- Break down complex questions into smaller steps in the `<think>` `</think>` section
- Answers are typically very short (1-3 words)
- If you fail at something, try again but think first

Available Tools:

1. Search for information:
`<search>` result = search('YOUR_SEARCH_QUERY')
`print(result)` `</search>`
2. Perform calculations using pandas, numpy, or math (no file I/O):
`<code>`# Your calculation code here
`print(result)` # ALWAYS print the result`</code>`

Question: **question.**

baseline single Large Language Model (LLM) call; an "Executor-only" agent equipped with code generation and search tools; an "Executor-Evaluator" agent, which augmented the executor with an evaluator component to assess task completion and guide iteration; and a full "Planner-Executor-Evaluator" multi-agent framework. The results of this phase guided the architecture choices for our primary investigation.

4.2 Main Experiments

Our experimental framework is designed to evaluate the capacity of language models to learn search engine integration for enhanced factual reasoning. We sample 200 questions from FlashRAG dataset Jin et al. (2024b) and mix them with 1k mathematical data samples from training split of Math Search-o1. We also sample 200 questions from FlashRAG and mix them with the test split of Math Search-o1 to serve as the in-domain testing dataset. The Testing Performance during training process is shown in Figure 2. To assess both in-domain and out-of-domain performance, we evaluate our models on a diverse set of additional benchmarks, including In-distribution (from the same source as training data), SimpleQA (Wei et al., 2024) for basic question answering, and the text-based subset of the GAIA benchmark (Mialon et al., 2023) for complex question answering.

Our main experiments focused on the Qwen-2.5-3B-base model (Yang et al., 2024), integrated with a search engine and code execution capabilities. We compared its performance with and without GRPO training.

For information retrieval, we employed a local dense retriever using the E5 model with FAISS indexing operating on the 2018 Wikipedia dump as the knowledge corpus. Following Lin et al. (2023), we set the number of retrieved passages to 3 for all retrieval-based methods. Our models were designed to support multi-turn search, with a maximum of two search turns, allowing for an interleaved process of reasoning and searching.

Training was conducted on a single node equipped with four H100 GPUs, using Fully Sharded Data Parallel (FSDP) with CPU offloading and gradient checkpointing to manage memory. We used a total batch size of 256. The maximum sequence length was set to 4,096 tokens. For efficient rollouts,

Table 2: Preliminary Results on DeepscaleR and MedQA

Benchmark	Single LLM	Executor-only	Executor+Evaluator	Planner+Executor+Evaluator
DeepscaleR (gpt-4.1-mini)	70	74	75	72
MedQA (gpt-4o-mini)	74	76	76	72

Table 3: Main Results on three held-out testing datasets

Method	In-distribution	SimpleQA	GAIA-text
Qwen-2.5-3B + search + code (w/o training)	0.01	0.08	0.00
CODESEARCH-R1(Qwen-2.5-3B-base w/ training)	0.26	0.24	0.04

we integrated vLLM with a tensor parallel size of 1, a GPU memory utilization of 0.6, a sampling temperature of 1.0, and a top-p of 1.0. For our primary reinforcement learning approach, Group Relative Policy Optimization (GRPO), we set the policy LLM learning rate to $1e-6$ and trained the model for 500 steps, using a learning rate warm-up ratio of 0.285. The KL divergence regularization coefficient β was set to 0.001 and the clip ratio ϵ to 0.2. Model checkpoints were saved every 100 steps, and we resumed from the most recent stable checkpoint if training diverged. Evaluation was based on the Exact Match (EM) metric. We directly compare the predicted answers to the ground truth using python functions during training and utilize Llama 4 Scout for comparing the predicted answers to the ground truth during testing.

5 Results

5.1 Preliminary Experiment Results: Agent Architecture Comparison

Our preliminary comparison of four distinct agent frameworks, summarized in Table 2, revealed that increasing architectural complexity does not guarantee better performance. The results clearly show that integrating code and search capabilities provides a foundational performance boost, with the Executor-only agent significantly outperforming the baseline Single LLM on both the DeepscaleR (74 vs. 70) and MedQA (76 vs. 74) datasets. While augmenting this with an evaluator component in the Executor+Evaluator framework yielded a marginal additional gain on DeepscaleR, it offered no further improvement on MedQA. This suggests that without specific reinforcement learning fine-tuning, the benefit of a self-assessment module is limited.

In contrast to the gains from tool integration, introducing an untrained high-level planner proved counterproductive. The most complex architecture, the Planner+Executor+Evaluator, resulted in a notable performance decrease to 72 on both benchmarks, falling behind even the simpler Executor-only agent. This indicates that for relatively straightforward tasks, the added overhead and potential for error from an unoptimized planner can hinder overall effectiveness. These findings led us to conclude that a streamlined agent architecture focusing on direct tool execution was the most promising and robust approach, guiding our design choices for the main experiments.

5.2 Main Experiment Results: CODESEARCH-R1 on Complex Tasks

Our main experiments confirm that reinforcement learning is a critical component in teaching language models to effectively utilize external tools for complex reasoning tasks. The results, presented in Table 3, show a stark contrast between the model’s capabilities with and without GRPO training.

The baseline model, Qwen-2.5-3B equipped with search and code execution tools but without any training, failed to leverage these capabilities effectively. It achieved near-zero performance across all benchmarks, with an Exact Match (EM) score of just 1% on the in-distribution data, 8% on SimpleQA, and 0% on the complex GAIA-text dataset. This demonstrates that merely providing access to tools is insufficient for improving reasoning performance.

In contrast, our proposed method, which applies GRPO training, unlocked the model’s potential and led to substantial gains. On the in-distribution test set, performance surged from 1% to 26% EM, showcasing the effectiveness of the training regimen. More importantly, the model demonstrated

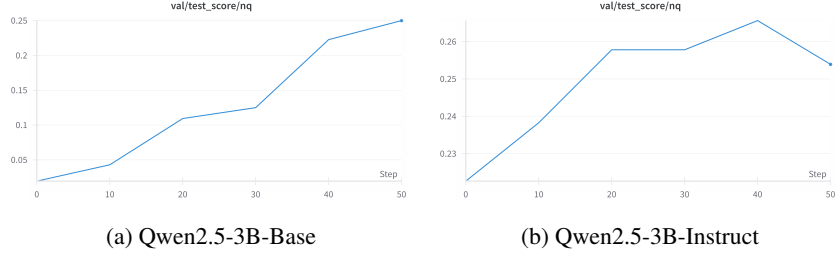


Figure 2: Testing Performance Curve.

strong generalization to out-of-domain datasets. It achieved a 24% EM on SimpleQA, tripling the baseline score. On the highly challenging GAIA-text benchmark, where the baseline completely failed, our trained model achieved a 4% EM score. While modest, this leap from zero indicates a newfound capability to begin tackling complex, multi-step reasoning problems.

Further analysis during training, as shown in Figure 2, justifies our choice of the base model over an instruct-tuned version. We found that the base model’s accuracy steadily increased from 0% to over 25% with training. The instruct model, however, started with a higher baseline accuracy (22%) but saw only marginal improvement before its performance declined. This suggests that the base model is more amenable to learning new behaviors through this style of reinforcement learning, whereas the instruct model’s prior fine-tuning may have hindered its adaptability. To provide qualitative insight into these improvements, we present successful cases of interleaved search and code execution in Tables 4 and 5, with failure cases analyzed in Appendix A.

Table 4: CODESEARCH-R1search and math example

Question: What is the square root of VAR_XTWI? Where: VAR_XTWI is obtained by getting the answer to this question/implementing the following coding problem: "who did the seahawks play in super bowl 2014?" Here is the guideline to help you format the answer: the answer consists of EXACTLY 2 words representing a proper team name Once you have found the answer to the problem, use the following formula to get the final value of VAR_XTWI: $\text{len}(\text{str}(\text{answer})) - 10$. The len has to be computed on the entire answer, that’s why we cast it to a string. Ground Truth: 2 CODESEARCH-R1: <code><think></code> to solve this i first need to find the value of VAR_XTWI, which is the length of the answer to the question "who did the seahawks play in super bowl 2014?" <code></think></code> <code><search></code> search("who did the seahawks play in super bowl 2014?") <code></search></code> <code><information></code> [1] the seahawks are playing against the denver broncos in super bowl 2014 ... <code></information></code> <code><think></code> ok so since i know that the answer is composed of words, the current answer is "denver broncos", let me compute the length of the answer and subtract 10 to get the final value of VAR_XTWI. <code></think></code> <code><code></code> <pre>k = len("denver broncos") - 10 print(k)</pre> <code></code></code> <code><information></code> 4 <code></information></code> <code><think></code> now there is additional information i need to find. in this case the square root of VAR_XTWI <code></think></code> <code><code></code> <pre>k = sqrt(4) print(k)</pre> <code></code></code> <code><information></code> 2 <code></information></code> <code><think></code> ok I have all the information I need, let me return the answer <code></think></code> <code><answer></code> 2 <code></answer></code>
--

5.3 Analysis

Effectiveness of Joint Code and Search Training Our results demonstrate that simultaneous training of code execution and search capabilities yields substantial improvements even with a 3B parameter model, achieving over 25% accuracy after just 50 training steps. This finding is particularly significant as it shows that effective tool integration is not exclusive to large-scale models. The synergistic effect between search and code execution enables complex problem-solving strategies, as evidenced in our case studies where the model successfully interleaves information retrieval with computational analysis.

We expect even greater performance gains with larger models, which typically excel at code generation and complex reasoning. The success on out-of-domain benchmarks like SimpleQA (24% accuracy) indicates that the model learns generalizable tool usage patterns rather than task-specific behaviors, suggesting robust transfer capabilities.

Base Models Superior for Tool Learning A striking finding is that base models significantly outperform instruction-tuned variants for tool learning through reinforcement learning. As shown in Figure 2, the base model exhibits steady improvement while the instruction-tuned model shows unstable training and inferior performance. This aligns with recent work suggesting that supervised fine-tuning creates rigid behavioral patterns that inhibit the exploration necessary for discovering effective tool usage strategies.

This has important implications for training pipelines: preserving base model flexibility appears crucial for multi-tool environments. The instruction tuning process likely introduces strong priors about response formats that conflict with the dynamic nature of tool-augmented reasoning, ultimately limiting the model’s potential for adaptation through RL.

6 Discussion

Our framework’s design enables exceptional flexibility through the code sandbox architecture, which theoretically supports any tool that can be called as a Python function. While we currently implement only search and code execution, this extensibility is crucial for specialized domains requiring custom tools such as molecular simulators, statistical packages, or domain-specific databases.

This architecture particularly suits our original goal of scientific research applications, where tasks require sophisticated integration of literature search, data analysis, and visualization. Scientists routinely iterate between searching for methodologies, implementing analyses, discovering requirements, and refining approaches—exactly the workflow our system supports. The promising results on general tasks suggest strong potential for scientific domains, though time constraints limited our evaluation to general benchmarks. Future work should focus on scientific datasets incorporating literature search, experimental analysis, and hypothesis testing to fully realize this potential.

7 Conclusion

In this work, we introduced and validated CODESEARCH-R1, a novel framework for training a single language model policy to master the integrated use of search engines and a code interpreter through reinforcement learning. Our experiments conclusively demonstrate that it is not mere access to tools, but end-to-end RL training with a simple, outcome-based reward that unlocks a model’s ability to perform complex, multi-step reasoning. We established that this streamlined, single-policy agent is more effective than complex, untrained multi-agent architectures and, critically, that a base LLM is far more adaptable to this training regimen than its instruction-tuned counterpart, suggesting that learning agentic skills is best achieved from a foundational model. The CODESEARCH-R1 framework therefore represents a significant step towards creating more autonomous and capable agents that learn robust problem-solving strategies from feedback, moving beyond the limitations of static prompting and rigid pipelines. While our work provides a strong proof-of-concept on general-domain benchmarks, the compelling next step is to apply this methodology to specialized domains like scientific research. Ultimately, CODESEARCH-R1 provides a validated methodology and a strong foundation for building the next generation of LLMs that do not just process information, but actively seek, compute, and reason with it to solve complex problems.

8 Team Contributions

- **Fan Nie** initiated the project idea, developed the initial codebase, and conducted key experiments.
- **Harper Hua** contributed to codebase development and experimental runs, and took primary responsibility for writing and poster preparation.
- **Mingjian Jiang** assisted in codebase implementation, carried out baseline experiments and helped with all the writing.

Changes from Proposal Our original hypothesis—exploration in reinforcement learning for language models—was overly broad and lacked a clear application context. We have since refined our objective to focus on developing a multi-role, tool-integrated reinforcement learning agent for scientific discovery, with an emphasis on search engine and coding tool use. This revised direction offers a more focused, practical, and compelling research trajectory that aligns better with real-world applications and our long-term goals.

References

- Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. 2018. Think you have solved question answering? try arc, the ai2 reasoning challenge. *arXiv preprint arXiv:1803.05457* (2018).
- Wei Fu, Jiakuan Gao, Xujie Shen, Chen Zhu, Zhiyu Mei, Chuyi He, Shusheng Xu, Guo Wei, Jun Mei, Jiashu Wang, Tongkai Yang, Binhang Yuan, and Yi Wu. 2025. AReaL: A Large-Scale Asynchronous Reinforcement Learning System for Language Reasoning. *arXiv:2505.24298 [cs.LG]* <https://arxiv.org/abs/2505.24298>
- Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Haofen Wang, and Haofen Wang. 2023. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997 2* (2023).
- Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shiron Ma, Peiyi Wang, Xiao Bi, et al. 2025. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948* (2025).
- Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. 2020. Measuring massive multitask language understanding. *arXiv preprint arXiv:2009.03300* (2020).
- Jie Huang and Kevin Chen-Chuan Chang. 2022. Towards reasoning in large language models: A survey. *arXiv preprint arXiv:2212.10403* (2022).
- Zhengbao Jiang, Frank F Xu, Luyu Gao, Zhiqing Sun, Qian Liu, Jane Dwivedi-Yu, Yiming Yang, Jamie Callan, and Graham Neubig. 2023. Active retrieval augmented generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 7969–7992.
- Bowen Jin, Jinsung Yoon, Jiawei Han, and Sercan O Arik. 2024a. Long-context llms meet rag: Overcoming challenges for long inputs in rag. In *The Thirteenth International Conference on Learning Representations*.
- Bowen Jin, Hansi Zeng, Zhenrui Yue, Jinsung Yoon, Sercan Arik, Dong Wang, Hamed Zamani, and Jiawei Han. 2025. Search-r1: Training llms to reason and leverage search engines with reinforcement learning. *arXiv preprint arXiv:2503.09516* (2025).
- Di Jin, Eileen Pan, Nassim Oufattole, Wei-Hung Weng, Hanyi Fang, and Peter Szolovits. 2020. What Disease does this Patient Have? A Large-scale Open Domain Question Answering Dataset from Medical Exams. *arXiv preprint arXiv:2009.13081* (2020).
- Jiajie Jin, Yutao Zhu, Xinyu Yang, Chenghao Zhang, and Zhicheng Dou. 2024b. FlashRAG: A Modular Toolkit for Efficient Retrieval-Augmented Generation Research. *CoRR* abs/2405.13576 (2024). <https://doi.org/10.48550/ARXIV.2405.13576> arXiv:2405.13576

- Aviral Kumar, Vincent Zhuang, Rishabh Agarwal, Yi Su, John D Co-Reyes, Avi Singh, Kate Baumli, Shariq Iqbal, Colton Bishop, Rebecca Roelofs, et al. 2024. Training language models to self-correct via reinforcement learning. *arXiv preprint arXiv:2409.12917* (2024).
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems* 33 (2020), 9459–9474.
- Zhuowan Li, Cheng Li, Mingyang Zhang, Qiaozhu Mei, and Michael Bendersky. 2024. Retrieval Augmented Generation or Long-Context LLMs? A Comprehensive Study and Hybrid Approach. arXiv:2407.16833 [cs.CL] <https://arxiv.org/abs/2407.16833>
- Muhan Lin, Shuyang Shi, Yue Guo, Vaishnav Tadiparthi, Behdad Chalaki, Ehsan Moradi Pari, Simon Stepputtis, Woojun Kim, Joseph Campbell, and Katia Sycara. 2025. Speaking the Language of Teamwork: LLM-Guided Credit Assignment in Multi-Agent Reinforcement Learning. arXiv:2502.03723 [cs.MA] <https://arxiv.org/abs/2502.03723>
- Xi Victoria Lin, Xilun Chen, Mingda Chen, Weijia Shi, Maria Lomeli, Richard James, Pedro Rodriguez, Jacob Kahn, Gergely Szilvasy, Mike Lewis, et al. 2023. Ra-dit: Retrieval-augmented dual instruction tuning. In *The Twelfth International Conference on Learning Representations*.
- Michael Luo, Sijun Tan, Justin Wong, Xiaoxiang Shi, William Y. Tang, Manan Roongta, Colin Cai, Jeffrey Luo, Li Erran Li, Raluca Ada Popa, and Ion Stoica. 2025. DeepScaleR: Surpassing O1-Preview with a 1.5B Model by Scaling RL. Notion Blog.
- Qianou Ma, Weirui Peng, Chenyang Yang, Hua Shen, Kenneth Koedinger, and Tongshuang Wu. 2025. What Should We Engineer in Prompts? Training Humans in Requirement-Driven LLM Use. *ACM Transactions on Computer-Human Interaction* (April 2025). <https://doi.org/10.1145/3731756>
- Grégoire Mialon, Clémentine Fourrier, Craig Swift, Thomas Wolf, Yann LeCun, and Thomas Scialom. 2023. GAIA: a benchmark for General AI Assistants. arXiv:2311.12983 [cs.CL] <https://arxiv.org/abs/2311.12983>
- Kartik Nagpal, Dayi Dong, Jean-Baptiste Bouvier, and Negar Mehr. 2025. Leveraging Large Language Models for Effective and Explainable Multi-Agent Credit Assignment. arXiv:2502.16863 [cs.MA] <https://arxiv.org/abs/2502.16863>
- Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. 2022. Training language models to follow instructions with human feedback. *Advances in neural information processing systems* 35 (2022), 27730–27744.
- M. H. Prince, H. Chan, A. Vriza, and et al. 2024. Opportunities for Retrieval and Tool Augmented Large Language Models in Scientific Facilities. , 251 pages. <https://doi.org/10.1038/s41524-024-01423-2>
- Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. 2023. Direct preference optimization: Your language model is secretly a reward model. *Advances in Neural Information Processing Systems* 36 (2023), 53728–53741.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems* 36 (2023), 68539–68551.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347* (2017).
- Maohao Shen, Guangtao Zeng, Zhenting Qi, Zhang-Wei Hong, Zhenfang Chen, Wei Lu, Gregory Wornell, Subhro Das, David Cox, and Chuang Gan. 2025. Satori: Reinforcement Learning with Chain-of-Action-Thought Enhances LLM Reasoning via Autoregressive Search. arXiv:2502.02508 [cs.CL] <https://arxiv.org/abs/2502.02508>

- Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. 2022. Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions. *arXiv preprint arXiv:2212.10509* (2022).
- Jason Wei, Nguyen Karina, Hyung Won Chung, Yunxin Joy Jiao, Spencer Papay, Amelia Glaese, John Schulman, and William Fedus. 2024. Measuring short-form factuality in large language models. *arXiv:2411.04368 [cs.CL]* <https://arxiv.org/abs/2411.04368>
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems* 35 (2022), 24824–24837.
- An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, et al. 2024. Qwen2. 5 technical report. *arXiv preprint arXiv:2412.15115* (2024).
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*.

A Appendix: Generation Examples

Table 5: CODESEARCH-R1search only successful example

Question: when did skiing halfpipe become an olympic event?
Ground Truth: 2014
CODESEARCH-R1:
<code><think></code> Let’s break down the question step by step and follow the guidelines: To answer this question, we need to search for the date when skiing halfpipe was added as an Olympic event. <code></think></code> <code><search></code> search('skiing halfpipe olympic event') <code></search></code> <code><information></code> [1] "Half-pipe skiing" Half-pipe skiing Half-pipe skiing is the sport of riding snow skis on a half-pipe. Competitors perform a series of tricks while going down the pipe. The current world record for highest jump in a half-pipe is held by Joffrey Pollet-Villard, with . The sport is considered to be dangerous compared to other sports, and helmets are required to be worn during competitions. Half-pipe skiing has been part of the Winter X Games since 2002, and made its Olympic debut at the 2014 Winter Olympics in Sochi, Russia. David Wise of the United States of America became the first Olympic champion [2] "Freestyle skiing at the 2018 Winter Olympics – Men’s halfpipe" also have a minimum of 50 FIS points to compete. If the host country, South Korea at the 2018 Winter Olympics did not qualify, their chosen athlete would displace the last qualified athlete, granted all qualification criterion was met. The top 12 athletes in the qualifiers moved on to the medal round. The final was held on 22 February 2018 at 12:22. Freestyle skiing at the 2018 Winter Olympics – Men’s halfpipe The men’s halfpipe event in freestyle skiing at the 2018 Winter Olympics took place on 20 and 22 February 2018 at the Bogwang Phoenix Park, Pyeongchang, South Korea. [3] "Snowboarding at the 2018 Winter Olympics – Men’s halfpipe" Snowboarding at the 2018 Winter Olympics – Men’s halfpipe The men’s halfpipe competition of the 2018 Winter Olympics was held from 13 to 14 February 2018 at the Bogwang Phoenix Park in Pyeongchang, South Korea. The event was won by Shaun White, who previously won this event in 2006 and 2010. Ayumu Hirano, the 2014 silver medalist, took silver. Scott James became the bronze medalist. For James, this is the first Olympic medal, and this was also the second medal for Australia at the 2018 Winter Olympics. In the victory ceremony, the medals were presented by John Dowling Coates, member’ <code></information></code> <code><think></code> The answer to the question can be found in the search results. <code></think></code> <code><answer></code> 2014 <code></answer></code>

Table 6: CODESEARCH-R1Failure Case-1

Question: who sang the song same as it ever was?
Ground Truth: Talking Heads
CODESEARCH-R1:
<code><answer></code> Fleetwood Mac <code></answer></code>

Table 7: CODESEARCH-R1 Failure Case-2

Question: Eugene swam on Sunday, Monday, and Tuesday. On Monday, he swam for VAR_GTDG minutes. On Tuesday, he swam for 45 minutes. His average swim time over the three days was 34 minutes. For how many minutes did he swim on Sunday? To find variables, answer the following questions: To get the value of the variable VAR_GTDG, answer the following question: What Armenian-founded manufacturer based in Massachusetts makes James Riley Breckenridge's cymbals? Note that the answer has the following guidelines: the answer is EXACTLY 3 words in the format FIRST_NAME LAST_NAME COMPANY_TYPE Once you have the answer, compute the following formula: $\text{len}(\text{str}(\text{answer})) + 7$. The len has to be computed on the entire answer, that's why we cast it to a string. to find the variable VAR_GTDG?

Ground Truth: 27

CODESEARCH-R1:

```
<think> To find the value of VAR_GTDG, we need to compute the length of the answer to the question "What
Armenian-founded manufacturer based in Massachusetts makes James Riley Breckenridge's cymbals?" and
add 7 to it. </think>
<code>
result = search('What Armenian-founded manufacturer based in Massachusetts makes James Riley Brecken-
ridge's cymbals?')
answer = result[0]
VAR_GTDG = len(str(answer)) + 7
print(VAR_GTDG)
</code>
<information> 8 </information>
<answer> 8 </answer>
```

Table 8: CODESEARCH-R1 Failure Case-3

Question: when did the nba 3 point line change?

Ground Truth: the 1997-98 season

CODESEARCH-R1:

```
<think> To answer this question, I need to search for information on when the NBA 3 point line changed.
</think>
<search> search('nba 3 point line change') </search>
<information> '[1] "Basketball court" line would give the player three free throws if the shot does not go
in, and one if it does. The distance to the three-point line from the center of the basket varies depending on
the level or league, and has changed several times. These are the current distances, with the league or level
using each distance: The NBA adopted the three-point line at the start of the 1979-80 season. This is of
variable distance, ranging from in the corners to behind the top of the key. During the 1994-95, 1995-96 and
1996-97 seasons, the NBA attempted to address decreased[2] "Three-point field goal" by to , with the change
being phased in beginning in October 2010. In December 2012, the WNBA announced that it would be using
the FIBA distance, starting in 2013; by 2017, the line at corners were lengthened to match the NBA. The
NBA has discussed adding a four-point line, according to president Rod Thorn. In the NBA, three-point field
goals have become increasingly more frequent along the years, with effectiveness increasing slightly. The
1979-80 season had an average 0.8 three-point goals per game and 2.8 attempts (29% effectiveness). The
1989-90 season had an average 2.2 three-point goals per game [3] "Basketball court" scoring by shortening
the overall distance of the line to a uniform around the basket. It was moved back to its original distance
after the 1996-97 season. FIBA and the NCAA both adopted the three-point line in 1985. In most high
school associations in the United States, the distance is 19.75 feet. This was formerly the distance for college
basketball as well. On May 26, 2007, the NCAA playing rules committee agreed to move the three-point line
back one foot to 20.75 feet for the men. This rule went into effect for the 2008-2009 season. The three-point
line for women' </information>
<think> From the search results, I can see that the three-point line in the NBA has been changed several
times. The current distance is 23.75 feet. </think>
<answer> 2010 </answer>
```